OMR Checker ! Using Open CV

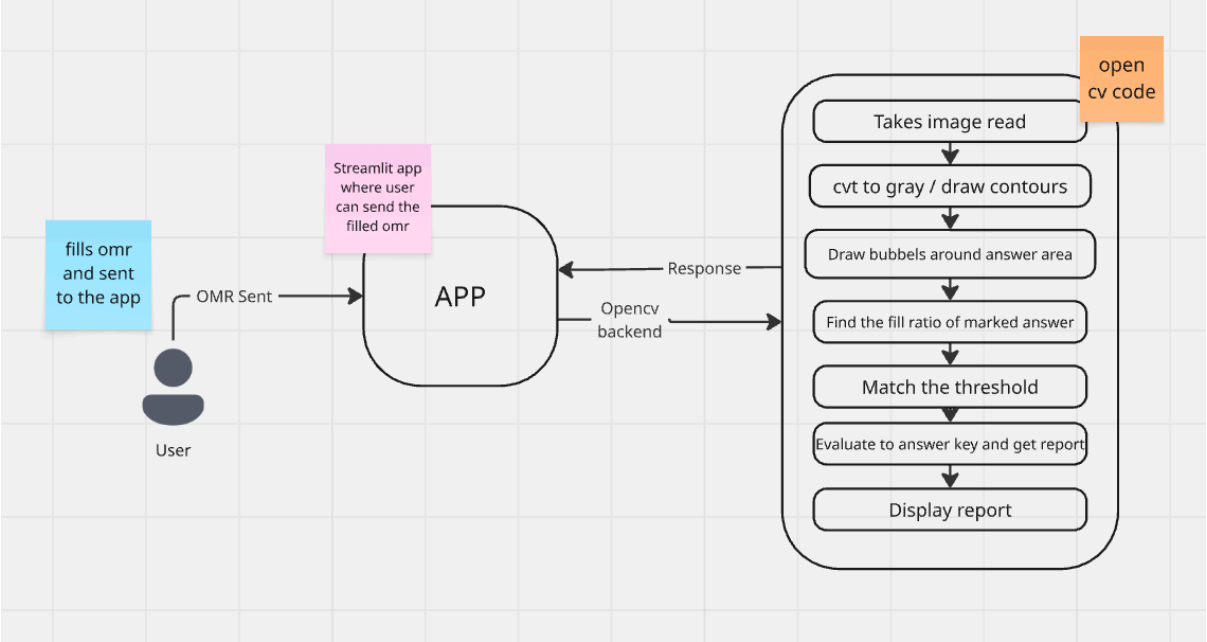
Abstract

Built a custom Optical Mark Recognition (OMR) evaluation system using computer vision. Inspired by my high school final exams, I designed this project to explore and replicate the automated grading mechanics behind physical OMR machines.

Project high level overview :

The system outlines an automated Optical Mark Recognition (OMR) processing flow where a user uploads a filled OMR sheet via a web interface, which is then processed by an OpenCV-based backend.

System Components & Data Flow

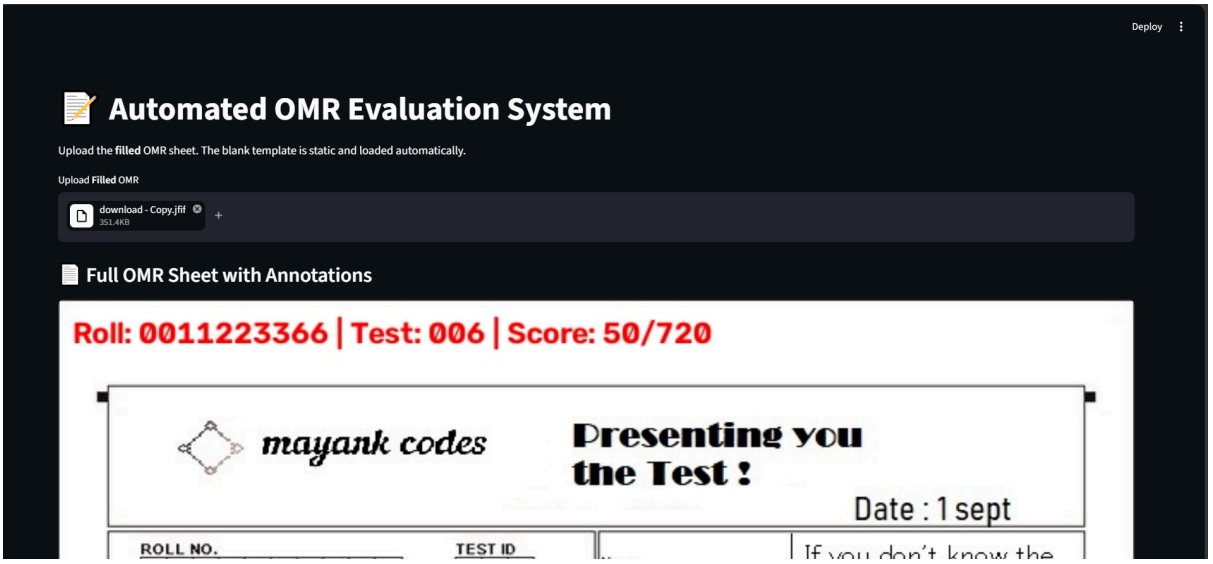


1. **User Interaction**
    - **Actor:** User
    - **Action:** The user physically completes/fills an OMR sheet and initiates the upload process.
2. **Frontend Interface (Streamlit)**
    - **Component:** Streamlit Web Application
    - **Description:** Provides an interactive UI for the user to upload or submit the filled OMR sheet image.
    - **Payload:** `OMR Sent` (The uploaded image file).
3. **Backend Processing (OpenCV Engine)**

- **Component:** OpenCV Backend (`Opencv backend`)
- **Description:** Receives the submitted OMR image from the Streamlit frontend to perform image processing, contour detection, and bubble/answer grid evaluation.

APP : Full Logic And Code Explanation

The OMR (Optical Mark Recognition) evaluation system is built using OpenCV for image processing and NumPy for numerical operations. The system follows a structured pipeline to extract and evaluate student responses from OMR sheets.



Pipeline

1. Image Preprocessing

```
blank_gray = cv.cvtColor(blank_img, cv.COLOR_BGR2GRAY)
```

```
filled_gray = cv.cvtColor(filled_img, cv.COLOR_BGR2GRAY)
```

- Converts both blank (template) and filled images to grayscale
- Reduces complexity while preserving structural information
- Enables threshold-based operations for bubble detection

2. Bubble Detection Algorithm

The system uses a multi-stage approach to identify filled bubbles:

```
def extract_grid_bubbles(area_gray, min_w=10, max_w=20, min_h=10,
max_h=20):

    _, thres = cv.threshold(area_gray, 200, 255, cv.THRESH_BINARY_INV)

        contours, _ = cv.findContours(thres, cv.RETR_EXTERNAL,
    cv.CHAIN_APPROX_SIMPLE)
```

Process:

1. Adaptive Thresholding: Converts grayscale to binary (dark bubbles become white)
2. Contour Detection: Identifies all potential bubble shapes
3. Geometric Filtering: Validates bubbles using:
    - Size constraints (10-20px width/height)
    - Aspect ratio (0.85-1.15 for circularity)
    - Area range (50-180 square pixels)
    - Circularity metric (0.4-1.3)

Circularity Calculation:

```
circularity = (4 * np.pi * area) / (perimeter * perimeter)
```

- Perfect circle: 1.0
- Accepted range: 0.4-1.3 (accounts for slight distortions)

3. Fill Ratio Analysis

```
def get_fill_ratio(image, x, y, radius=5):

    roi = image[y - radius : y + radius + 1, x - radius : x + radius + 1]

    mask = np.zeros(roi.shape, dtype=np.uint8)

    cv.circle(mask, center, radius, 255, -1)

    pixels = roi[mask == 255]
```

```
    dark_pixels = np.sum(pixels < 150)

    return dark_pixels / len(pixels)
```

Process:

1. Extracts a circular region of interest (ROI) around each bubble center
2. Creates a circular mask to exclude corners
3. Counts pixels darker than threshold (150)
4. Computes ratio = dark_pixels / total_pixels_in_circle

Decision Logic:

- `fill_ratio >= 0.35` → Bubble is considered FILLED
- `fill_ratio < 0.35` → Bubble is considered EMPTY

4. Grid Organization

```
def organize_into_rows(bubbles, y_threshold=5):

    bubbles = sorted(bubbles, key=lambda p: p[1])  # Sort by Y coordinate

    # Group into rows based on Y proximity

        # Sort each row by X coordinate (left to right)
```

Process:

1. Row Detection: Sorts bubbles by Y-coordinate, groups those within 5px
2. Column Sorting: Within each row, sorts by X-coordinate
3. Grid Formation: Creates a structured 2D grid of bubble positions

5. Roll Number & Test ID Extraction

```
def parse_digit_grid(blank_crop, filled_crop):

    centers = extract_grid_bubbles(blank_crop)  # Get template positions

    rows = organize_into_rows(centers)          # Organize into grid

    # For each column, find which row is filled
```

```
    # Convert row index to digit value
```

Process:

- Uses blank template to identify bubble positions
- For each column, checks which row is filled (0-9)
- Converts row index to digit (0-9)
- Handles special cases:
  - Single filled: returns digit
  - Multiple filled: returns "X"
  - No filled: returns "?"

6. Answer Extraction

```
questions = get_question_bubbles(rows)
```

```
# Maps each question number to its 4 option bubbles
```

Structure:

- 180 questions organized in a 6x30 grid
- Each question has 4 options (A, B, C, D)
- Option mapping: [A, B, C, D] → [0, 1, 2, 3]

Extraction Logic:

1. For each question, examine its 4 bubbles
2. Identify filled options using fill ratio threshold
3. Classify as:
   - Single: Valid answer (one option filled)
   - Multiple: Invalid (more than one option filled)
   - Blank: No option filled

7. Evaluation & Scoring

```
for q_num, correct_ans in ANSWER_KEY.items():

    ans = student_answers.get(q_num, "BLANK")

    if ans == "BLANK":      # MARKS_BLANK = 0
```

```
elif ans == "MULTIPLE": # MARKS_MULTIPLE = 0

elif ans == correct_ans: # MARKS_CORRECT = +4

    else:                       # MARKS_WRONG = -1
```

Scoring Scheme:

- Correct Answer: +4 marks
- Wrong Answer: -1 mark (negative marking)
- Blank: 0 marks
- Multiple Answers: 0 marks (considered invalid)

Performance Metrics:

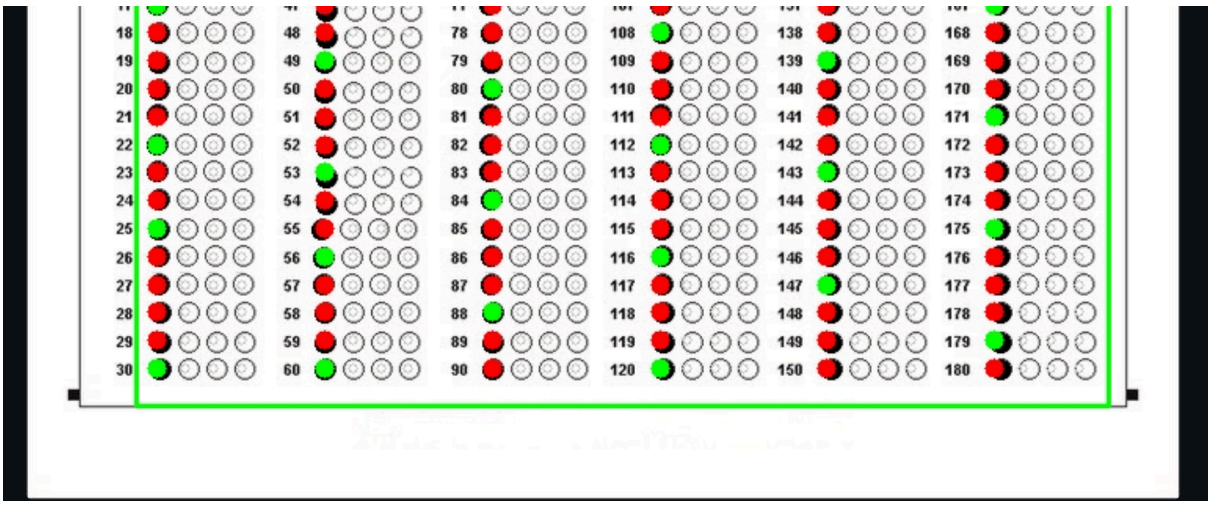
- Total Score: Sum of all question marks
- Maximum Marks: 180 × 4 = 720
- Percentage: (Total Score / 720) × 100

8. Visualization & Debugging

```
# Annotate each bubble

if option_char == student_ans:

    color = (0, 255, 0) if student_ans == correct_ans else (0, 0, 255)

    cv.circle(debug_display, (x, y), 6, color, -1)
```

Visual Indicators:

- Green Circle: Correct answer
- Red Circle: Wrong answer
- Gray Circle: Unmarked option
- Green Rectangle: Answer area boundary



Data Structures

ANSWER_KEY Dictionary

```
ANSWER_KEY = {
    1: "C", 2: "A", 3: "D", ...  # 180 questions
}
```

question_details Dictionary

```
{
    question_number: {
        'student_answer': 'A' | 'B' | 'C' | 'D' | 'BLANK' | 'MULTIPLE',
        'correct_answer': 'A' | 'B' | 'C' | 'D',
        'status': 'Correct' | 'Wrong' | 'Blank' | 'Multiple',
        'marks': 4 | -1 | 0
    }
}
```

Image Coordinate System

The system uses fixed pixel coordinates for different sections:

Section	X Range	Y Range
Roll Number	35-240	155-380
Test ID	250-360	155-380
Answer Area	70-690	400-980

Key Algorithms Summary

1. Contour Detection: Finds bubble shapes using OpenCV's `findContours`
2. Geometric Validation: Filters bubbles based on size, shape, and circularity
3. Threshold Analysis: Determines filled/unfilled state using pixel intensity
4. Grid Mapping: Organizes bubbles into rows and columns
5. Pattern Matching: Matches student answers against answer key
6. Score Computation: Calculates marks with negative marking scheme

---

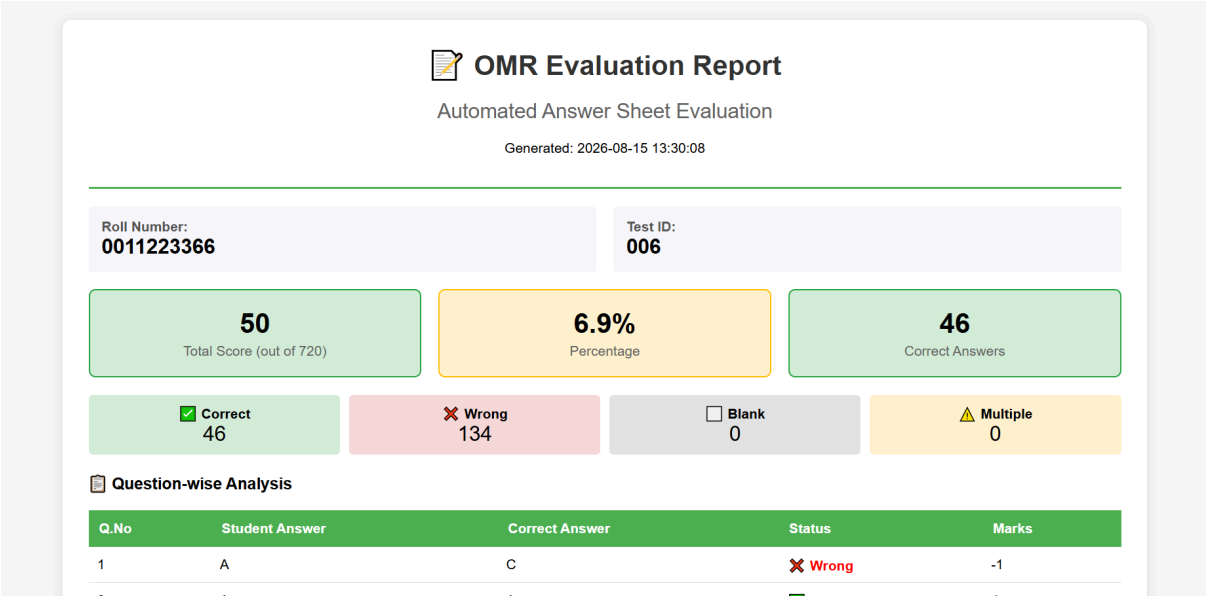
Features

📝 Core Evaluation Features

- Automated Bubble Detection: Uses computer vision to identify filled bubbles
- Multi-Section Analysis: Extracts Roll Number, Test ID, and 180 answers
- Smart Marking Scheme: Supports +4 for correct, -1 for wrong answers
- Error Handling: Detects invalid markings (multiple options, blank responses)
- Real-time Processing: Instant evaluation of uploaded OMR sheets

📊 Reporting & Visualization

- Full OMR Display: Shows entire answer sheet with visual annotations
- Color-Coded Feedback: Green for correct, red for wrong answers
- Performance Dashboard: Displays score, percentage, and breakdown
- Question-wise Analysis: Detailed view of each question's status
- Interactive Data Table: Sortable and searchable results



📥 Export Capabilities

- HTML Report: Professional, printable evaluation report

- CSV Marksheet: Spreadsheet-compatible detailed analysis
- Annotated Image: Download marked OMR sheet with visual feedback

🎯 User Experience

- Intuitive Interface: Clean Streamlit-based web application
- Single Upload: Only requires filled OMR sheet (blank template is static)
- Real-time Results: Immediate feedback after upload
- Mobile Responsive: Works on different screen sizes
- Error Feedback: Clear messages for invalid uploads or processing errors

🔧 Technical Advantages

- OpenCV-based: Robust image processing engine
- NumPy Integration: Efficient numerical computations
- Scalable Architecture: Can handle multiple sheets
- Modular Code: Easy to extend or modify
- Cross-Platform: Runs on any system with Python

📈 Performance Metrics

- Accuracy Tracking: Reports correct, wrong, blank, and multiple counts
- Percentage Calculation: Shows overall performance percentage
- Score Analysis: Detailed breakdown of marks distribution
- Max Score Display: Shows total possible marks (720)

🔒 Data Management

- Timestamped Reports: All downloads include date-time stamps
- Structured Data: Organized CSV and HTML formats
- Preserved Metadata: Roll number and Test ID included in all reports
- Non-destructive Processing: Original images remain unchanged

---

Technical Requirements

- Python Libraries: OpenCV, NumPy, Pandas, Streamlit
- Image Formats: PNG, JPG, JPEG
- Output Formats: HTML, CSV, PNG
- Memory Usage: ~100MB per evaluation
- Processing Time: <1 second per OMR sheet

Future Enhancements

1. Batch Processing: Support for multiple OMR sheets at once
2. Database Integration: Store results for multiple students
3. Custom Answer Keys: Support for different test configurations
4. OCR Integration: Extract text from OMR sheets
5. Statistical Analysis: Class performance analytics
6. Export to PDF: Enhanced report generation
7. Cloud Deployment: Web-based evaluation service
8. Mobile Application: Phone-based OMR evaluation